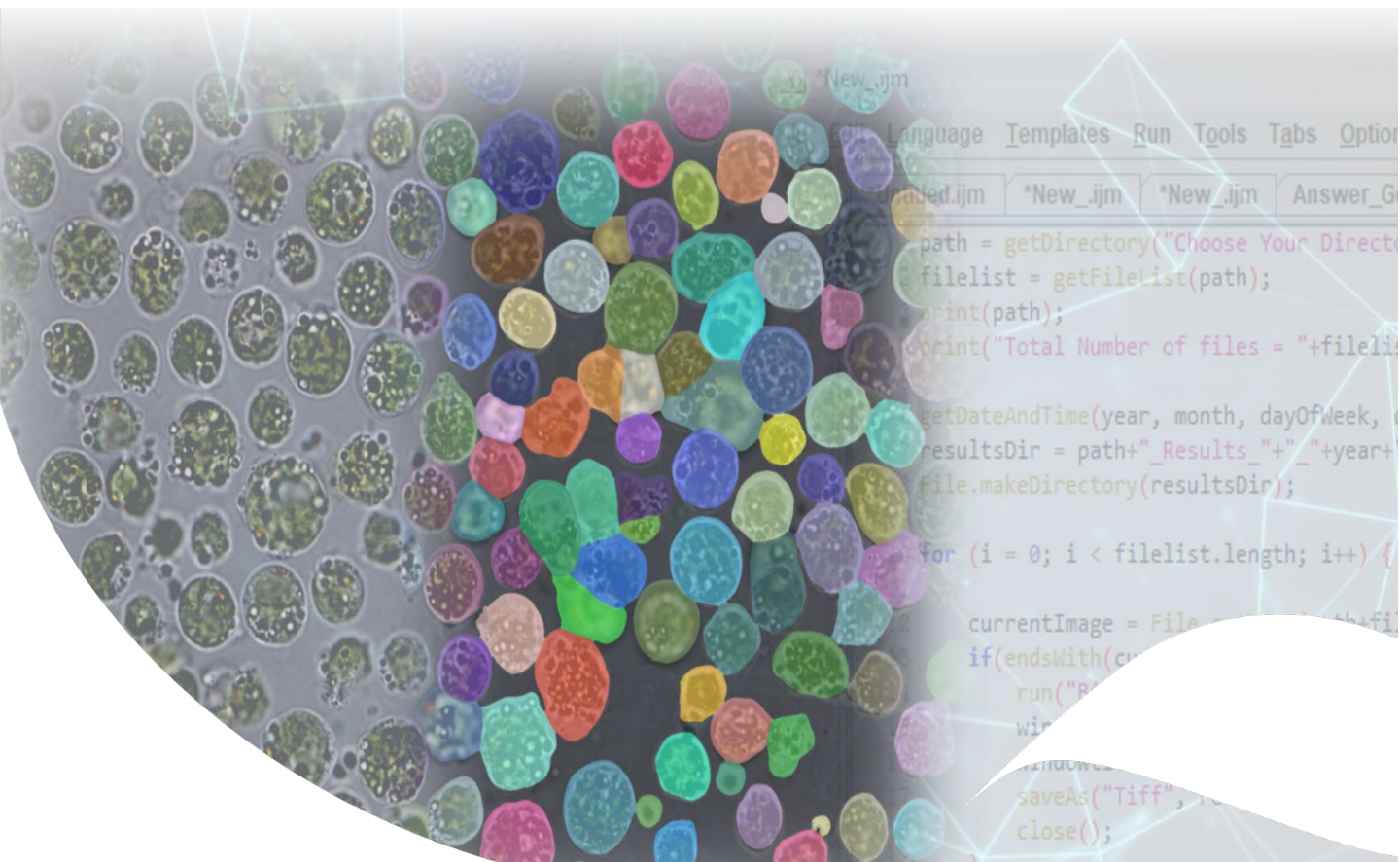


Introduction to Macro Writing in Fiji



Presented by
Dr Nicholas Condon

Name: _____



Introduction to Macro Writing in Fiji

August 2026

Dr Nicholas Condon

n.condon@uq.edu.au

Acknowledgement of Country

The University of Queensland (UQ) acknowledges the Traditional Owners and their custodianship of the lands on which we meet.

We pay our respects to their Ancestors and their descendants, who continue cultural and spiritual connections to Country.

We recognise their valuable contributions to Australian and global society.

The Brisbane River pattern from *A Guidance Through Time*
by Casey Coolwell and Kyra Mancktelow.



Workshop structure

1

Background

2

Essential Knowledge

3

Getting Started in Fiji

4

Programming basics – Conditional
Code

5

Scripting in Fiji

6

Structuring your macros

7

Coding Tips & Tricks

Scripting in Fiji, why automate?

- Microscopes can generate lots of data!
- More complex studies are required (screens, panels etc)
- Helps speed up analysis
- Streamlines complex workflows
- Reduces bias
- It makes your life easier (sometimes)!
- Can support multiple languages (this workshop only covers IJM)
- Often can be published as a follow-up methods paper/book chapter.

```
1
2 path = getDirectory("Choose Source Directory ");
3 list = getFileList(path);
4 //setBatchMode(true);
5 resultsDir = path+"Results/";
6 File.makeDirectory(resultsDir);
7
8 for (i=0; i<list.length; i++) {
9   if (endsWith(list[i],".tif")){
10
11
12     open(path+list[i]);
13     windowtitle = getTitle();
14     run("Make Composite");
15     run("Split Channels");
16     run("Clear Results");
17
18     selectWindow("C2-"+windowtitle);
19     close();
20
21     selectWindow("C3-"+windowtitle);
22     rename("DAPI");
23     run("Subtract Background...", "rolling=100");
24     run("Find Maxima...", "noise=2 output=[Single Points]");
25     rename("Points");
26     setOption("BlackBackground", false);
27     run("Dilate");
28     run("Dilate");
29     run("Analyze Particles...", "add");
30
31     selectWindow("C1-"+windowtitle);
32     rename("Marker");
33     run("Subtract Background...", "rolling=100");
34     run("Median...", "radius=2");
35     run("Threshold...");
```

Is it worth it

- Scripting does take time, versioning and updating
- Development therefore requires a cost-benefit analysis.
- Think about how much time it will save, not just once but every time you need to run it.

HOW LONG CAN YOU WORK ON MAKING A ROUTINE TASK MORE EFFICIENT BEFORE YOU'RE SPENDING MORE TIME THAN YOU SAVE?
(ACROSS FIVE YEARS)

		HOW OFTEN YOU DO THE TASK					
		50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
HOW MUCH TIME YOU SHAVE OFF	1 SECOND	1 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
	5 SECONDS	5 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
	30 SECONDS	4 WEEKS	3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
	1 MINUTE	8 WEEKS	6 DAYS	1 DAY	4 HOURS	1 HOUR	5 MINUTES
	5 MINUTES	9 MONTHS	4 WEEKS	6 DAYS	21 HOURS	5 HOURS	25 MINUTES
	30 MINUTES		6 MONTHS	5 WEEKS	5 DAYS	1 DAY	2 HOURS
	1 HOUR		10 MONTHS	2 MONTHS	10 DAYS	2 DAYS	5 HOURS
	6 HOURS				2 MONTHS	2 WEEKS	1 DAY
	1 DAY					8 WEEKS	5 DAYS

<https://xkcd.com/1205/>
Used under CC:2.5

LM@UQ WS7 - Fiji Macros | Aug 2026

Overview



Essential knowledge

- Data Types
- Variables
- Operations
- Functions
- Conditional Statements
- Loops



How to apply it in Fiji

- Script creation interface
- Useful functions
- Common Pitfalls (for Fiji script)
- Lots of common examples
- Tips & Tricks



Best Practices

- Commenting your script
- Pseudocode your pipeline
- Write the code
- Validate your code
- Debugging your code

LM@UQ WS7 - Fiji Macros | Aug 2026

2 Essential Knowledge

All the dry, but important
parts of coding

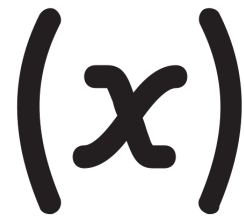
Data types

How we describe our data

Type	Examples	Used for?
Integer	0,1,2,... 0,1,2,...255 0,1,2,...65535	Whole numbers, indexing Intensity value (8-bit) Intensity value (16-bit)
Signed Integer	...-2, -1,0,1,2,...	Whole numbers, indexing
Float	-123.45, 0.0, 6.789, etc.	Calculation results
Boolean	TRUE or 1 FALSE or 0	Debugging multiple criteria in complicated conditional statements
Character	A, a, B, b, etc.	Building a word...
String	"Microscopy is how you say, fun?"	Holds an array of characters

Variables

How do we hold and access information?



- **A variable (in FIJI) is a container of memory that:**
 - Holds data
 - Can be referred to by name
 - Will be written over if re-used or re-defined throughout your code
 - Does not care what data type gets assigned (i.e. integer, string, video game cheat codes, etc.)
 - **GOOD:** Make a new variable as you need one, no pre-declaration required
 - **BAD:** If you change the data type via assignment (i.e. overwrite a string with a float), you can break subsequent code

LM@UQ WS7 - Fiji Macros | Aug 2026

Naming variables

You need a thing, give it a name

- **Variable names:**
 - Are case-sensitive (i.e. myvariable is different from myVariable)
 - Cannot contain symbols or spaces
 - Can be as short as one letter (not a good idea) or very verbose (also not a good idea)
 - Should not be function names (run, print etc.) or keywords (do, while, if, for, etc.)
 - Should consider some **“old school”** naming conventions

Example	Meaning
fMyVariable	Variable holds a float value
iMyVariable	Variable holds an integer value
sMyVariable	Variable holds a string (a.k.a. array of chars)
aMyArray	Variable is an array of data
idxMyArray	Index into an array of data

Be clear about the purpose of your variables!
 Name your variable based on the purpose
 Your future self will thank you when the new person in the lab (but more likely, **you**) breaks your code!

LM@UQ WS7 - Fiji Macros | Aug 2026

Operations

How to do 'stuff' to your variables

- **LEFT SIDE is variable name:**

- iNumNucleus
- sMyFolder

- **RIGHT SIDE is the value:**

- 100 (integer)
- "My_name" (string)
- 5.6 (float)

- **Example:**

- iNumNucleus = 100;
- sMyFolder = "My_name";

LM@UQ WS7 - Fiji Macros | Aug 2026

=	Assigning a value to a variable
---	---------------------------------

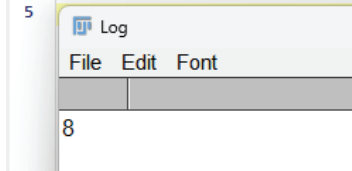
Operations – Basics

Both numbers (integers) and sometimes words (strings) can be used for operations (to different effects)

- **Integers:**

- If you need to perform a calculation standard computational operators can be used

```
1 score1 = 3;
2 score2 = 5;
3 totalScore = score1+score2;
4 print(totalScore);
5
```

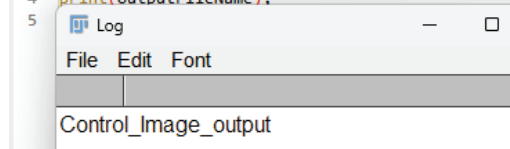


- **Strings:**

- Sometimes you may want to concatenate strings, in Fiji you can use the addition operator to do this easily

Operator	Description
+	Addition
-	Minus
*	Multiplication
/	Division

```
1 inputFilename = "Control_Image"
2 outputSuffix = "_output"
3 outputFile = inputFilename+outputSuffix;
4 print(outputFile);
5
```



LM@UQ WS7 - Fiji Macros | Aug 2026

Arrays

Holding a list of variables

- Arrays & lists are interchangeable in Fiji
- Denoted by []
- *Items* are separated by commas
- Array indexing starts at 0
We call this Base-0

Name	Vendor	Room	Type
Confocal 1	Leica	6029A	Confocal
Zeiss Live Imager 2	Zeiss	6025A	Automated Widefield
Confocal 2	Zeiss	6029	Confocal
Dragonfly	Andor	6025C	Spinning Disc Confocal
Fluoro1	Olympus	3023	Widefield

How to make the first 2 columns into arrays?

Name = [“Confocal 1”, “Zeiss Live Imager 2”, “Confocal 2”, “Dragonfly”, “Fluoro1”];

Vendor = [“Leica”, “Zeiss”, “Zeiss”, “Andor”, “Olympus”];

Arrays

How do you get info out of an array?

Individual items in an array can be recalled by querying their index

Whole array statistics can also be queried (i.e. length, max, min etc)

Name	Vendor	Room	Type
Confocal 1	Leica	6029A	Confocal
Zeiss Live Imager 2	Zeiss	6025A	Automated Widefield
Confocal 2	Zeiss	6029	Confocal
Dragonfly	Andor	6025C	Spinning Disc Confocal
Fluoro1	Olympus	3023	Widefield

E.g. **Name[1]** is “Zeiss Live Imager 2”
Room[2] is “6029”

However, if you want to know everything about Zeiss Live Imager 2, use index 1 from each array

SystemName = Name[1];
SystemVendor = Vendor[1];
RoomLocation = Room[1];
SystemType = Type[1];

SystemName = “Zeiss Live Imager 2”
SystemVendor = “Zeiss”
RoomLocation = “6029A”
SystemType = “Automated Widefield”

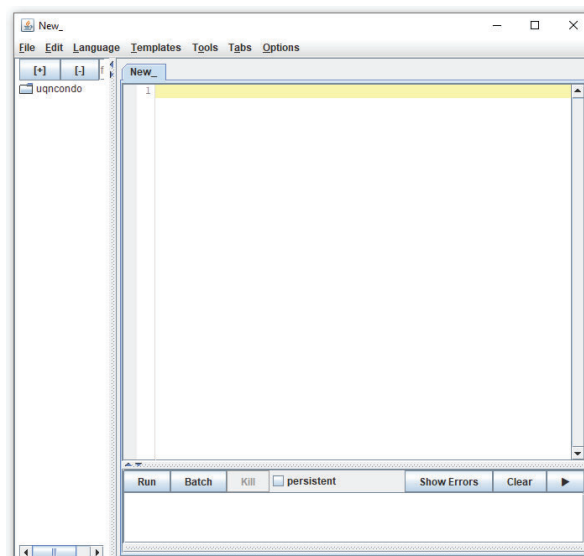
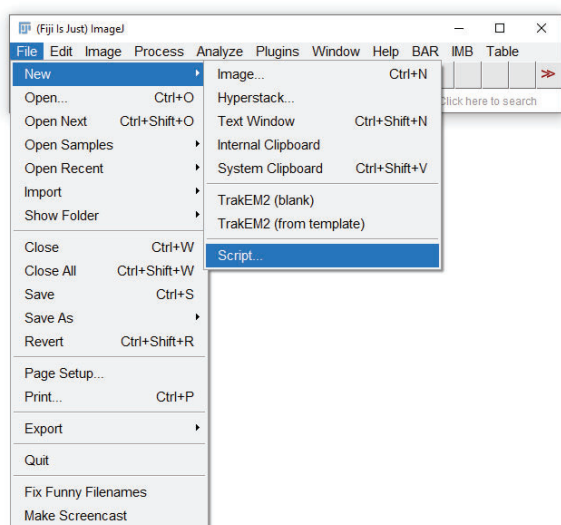
3 Getting started in Fiji

Ensure you have updated
Fiji recently for the best
quality of life

Launching the Macro/Scripting Window

File > New > Script...

Keyboard shortcut “[



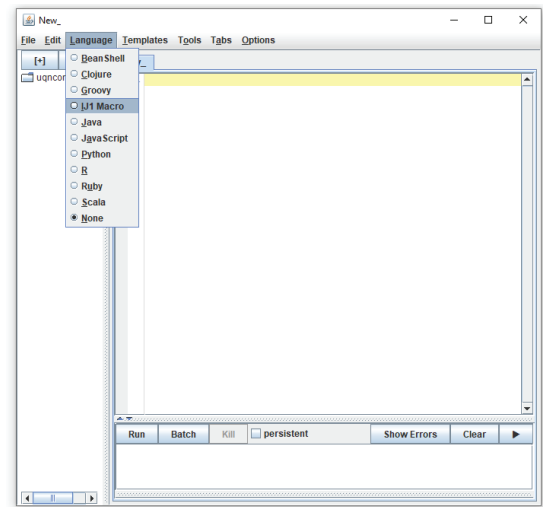
Setting up your IDE

Choosing the language

There are many supported languages in Fiji:

- BeanShell
- Clojure
- Groovy
- **IJ1 Macro**
- Java
- JavaScript
- Python
- R
- Ruby
- Scala

Pro Tip: there are inbuilt IJ1 functions for lots of things including arrays and image selections
<https://wsr.imagej.net/developer/macro/functions.html>



LM@UQ WS7 - Fiji Macros | Aug 2026

Basic syntax

Colour and styling for readability

`// comments to describe your code`

Double forward slash: one-line comment, not for execution

`; semicolon`

Syntax for end of current line to execute

`numericalVariable = 5;`

Blue: An explicit value

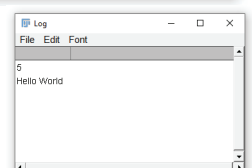
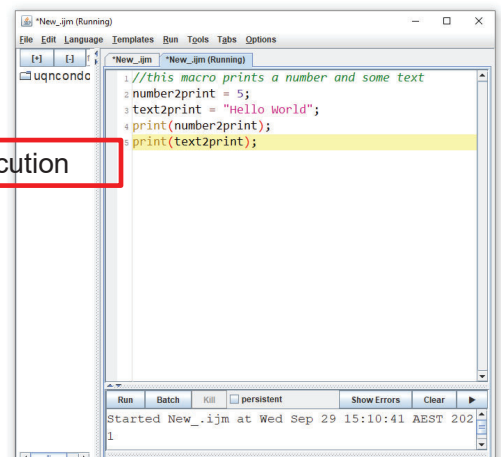
`stringVariable = "text";`

Pink: An array of chars within the double quotation marks

`builtInCommand([arg]);`

Gold: A function (with optional arguments)

More detail on this in next section!



LM@UQ WS7 - Fiji Macros | Aug 2026

Functions

Built in commands, some need no input, others require more information (**called Arguments**)

No Arguments

```
getTime();
selectionType();
```

Asking FIJI for a state value – doesn't need an input

Single Arguments

```
roiManager("count");
setSlice(n);
```

Simple function that only needs one input from user

Multiple Arguments

```
run("Median...", "radius=2");
run("Z Project...", "projection=[Max Intensity]");
run("Z Project...", "start=50 stop=100 projection=[Max Intensity]");
```

Some functions can change their behaviour based upon
(i) multiple input arguments
(ii) mandatory versus optional input

Use the "Z Project" macro to return the maximum intensity projection from index 50 to 100

LM@UQ WS7 - Fiji Macros | Aug 2026

Arrays in Fiji

Arrays are one dimensional tables or lists and can hold integers or strings.

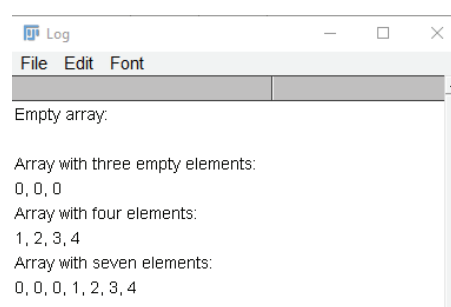
```
emptyArray = newArray(0);
print("Empty array:");
Array.print(emptyArray);

arrayWithThreeElements = newArray(3);
print("Array with three empty elements:");
Array.print(arrayWithThreeElements);

arrayWithFourElements = newArray(1,2,3,4);
print("Array with four elements:");
Array.print(arrayWithFourElements);

longArray = Array.concat(arrayWithThreeElements, arrayWithFourElements);
print("Array with seven elements:");
Array.print(longArray);
```

LM@UQ WS7 - Fiji Macros | Aug 2026

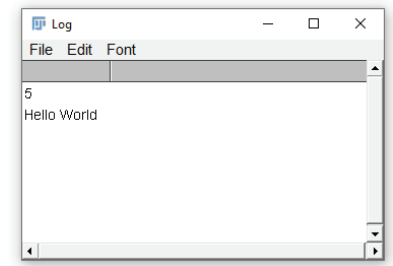


Pro Tip: `nameofarray.length` returns the length of the array
e.g. `longArraylen = longArray.length`
Gives `longArraylen = 7`

Pro Tip:
`setOption("ExpandableArrays", true);`
Returns lets arrays increase in size.

Printing to the log window

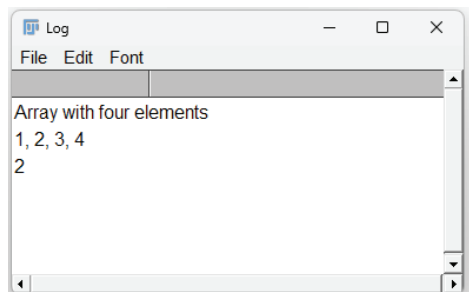
The print command can write information out to the log window.
It can be used to state something, or can be useful when debugging your code.



- `Print();` `//prints an empty line`
- `print("Hello World");` `//prints a defined string`
- `String="Hello World"; Print(string);` `//prints a variable string`
- `Print(nSlices);` `//resolves argument & prints result`
- `print("\\Clear");` `//clears all lines printed in the log window`
- `print("Number of slices =" + nSlices);` `//prints string and resolved argument`

Printing with Arrays

Printing a whole array requires a special function, printing an element of an array uses the normal print function.



```
arrayWithFourElements = newArray(1,2,3,4);
print("Array with four elements:");
Array.print(arrayWithFourElements);
print(arrayWithFourElements[1]);
```

Quick Coding Guide

Arrays

```
myArray = newArray(0);  
longArray = Array.concat(arr1, arr2)  
Array.print(arr1)
```

Functions

```
path = getDirectory("Choose Your  
Directory");  
filelist = getFileList(path);  
resultsDir = path+"_Results";  
File.makeDirectory(resultsDir);
```

Resetting

```
close();  
run("Clear Results");  
roiManager("reset");  
while(nImages>0){close();}
```

Print statements

```
print();  
print(myResult);  
print(nSlices);  
print("\\Clear");
```

LM@UQ WS7 - Fiji Macros | Aug 2026

Conditional statements

```
if (condition = true) { // Do code  
for (i=0; i<10; i++) { // Do code  
while (condition) { // Do code
```

Image information

```
windowTitle = getTitle();  
selectWindow(windowTitle);  
getDimensions(width, height, channels, slices,  
frames);  
nSlices;  
getPixelSize(unit, pixelWidth, pixelHeight);  
getVoxelSize(width, height, depth, unit);  
getResult("column", row);
```

Saving & Reporting

```
saveAs("Tiff", resultsDir+"myfilename");  
summaryFile = File.open(resultsDir+"Results.csv");  
print(summaryFile, "Image Name, ... Results");
```

ROI Manager

```
roiManager("add");  
roiManager("delete");  
roiManager("select", index);  
roiManager("measure");  
roiManager("rename", name);  
roiManager("count");  
roiManager("reset");
```

Common Functions

```
run("Median...", "radius=2");  
run("Z Project...", "projection=[Max  
Intensity]");  
run("Duplicate, ...");  
run("Duplicate...",  
"title="+windowTitle+"_copy");  
run("Split Channels");  
run("Measure");  
myAreaResult = getResult("Area", 0);  
setAutoThreshold("Huang no-reset");  
run("Convert to Mask");
```

Operations - Advanced

Additions

Operator	Description
++	pre or post increment
--	pre or post decrement
-	minus
!	boolean complement
~	bitwise complement
*	multiplication
/	division
%	remainder
&	bitwise AND
	bitwise OR
^	bitwise XOR
<<, >>	left shift, right shift
+	addition or string concatenation
-	subtraction
<, <=	less than, less than or equal
>, >=	greater than, greater than or equal
==, !=	equal, not equal
&&	boolean AND
	boolean OR
=	assignment
+=, -=, *=, /=	assignment with operation

+	addition or string concatenation
---	----------------------------------

• ASSUME:

A = 1, B = 2, C = 3, D = 0, F = "Maybe"

ADDITION:

IntegerResult = A + B;
= 3

StringResult = "Carly Rae Jeppsen one hit
wonder is Call Me " + F;

Note: Plus sign will "concatenate" strings!

Exercise 1: Using the Print Command

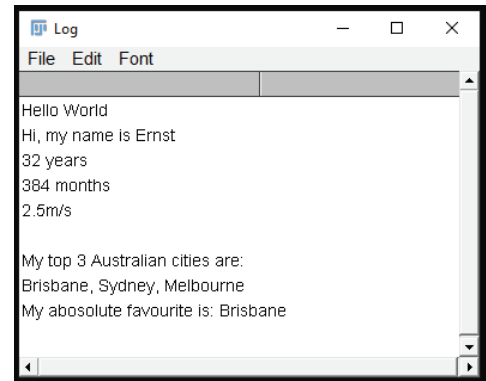
Open and run a Fiji script to help familiarise yourself with this tool to speed up your workflow.
Use your imagination for some of these concepts.

5
Minutes

Task:

Use the print command to experiment with the following:

- **Strings:** e.g. "Hello World"
- **Concatenated string:** e.g. Hi, my name is +<variable>
Array.concat(string1, string2)
- **Integers:** e.g. Your Age (in years)
- **Calculated Integers:** e.g. your age in months (years * 12)
- **Calculated Float:** e.g. calculate speed (first declare 2 variables then calculate the answer in the print statement)
- **Create an array** with your top 3 favourite cities & print it
- **Print** the first element of that array as your favourite.



Remember `print()` ;

LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 1 - Answers

1. Clear the log window
2. Print the string
3. Define a variable, print a string and concatenate the variable to it
4. Print a number
5. Print the calculated result of your age * 12 and add the string to the end.
6. Define distance and time as variables, print the calculation
7. Assign an array with 3 strings as the elements
8. Print the entire array
9. Print only one element of that array

```
Exercise1_Answers.ijm  New_ijm
1 //resets log window
2 print("\Clear");
3
4 //prints defined string
5 print("Hello World");
6
7 //prints variable
8 usrName = "Ernst";
9 print("Hi, my name is "+usrName);
10
11 //prints integer + string together
12 print(32+" years");
13
14 //performs calculation and prints integer and string together
15 print(32*12+" months");
16
17 //defines two variables, performs calculation and prints resulting integer + string
18 distance = 100;
19 time = 40;
20 print(distance/time+ "m/s");
21
22 //prints a spacer
23 print("");
24
25 //defines array and prints both whole array and single array element
26 cities = newArray("Brisbane", "Sydney", "Melbourne");
27 print("My top 3 Australian cities are:");
28 Array.print(cities);
29 print("My absolute favourite is: "+cities[0]);
```

LM@UQ WS7 - Fiji Macros | Aug 2026

Operations - Advanced

Division

Operator	Description
++	pre or post increment
--	pre or post decrement
-	minus
!	boolean complement
~	bitwise complement
*	multiplication
/	division
%	remainder
&	bitwise AND
	bitwise OR
^	bitwise XOR
<<, >>	left shift, right shift
+	addition or string concatenation
-	subtraction
<, <=	less than, less than or equal
>, >=	greater than, greater than or equal
==, !=	equal, not equal
&&	boolean AND
	boolean OR
=	assignment
+=, -=, *=, /=	assignment with operation

/	Division
---	----------

• ASSUME:

A = 1, B = 2, C = 3, D = 0, F = "Maybe"

DIVISION:

Result = C/B
= 1.5

Some script languages might truncate to 1 when assigning a float result to a previously defined integer variable!

Assume the text parser is a Golden Retriever, so if you want your result to have a decimal (be a float) make sure B=2.0 and C=3.0

LM@UQ WS7 - Fiji Macros | Aug 2026

Operations - Advanced

Logical Tests

Operator	Description
<, <=	less than, less than or equal
>, >=	greater than, greater than or equal
==, !=	equal, not equal
&&	boolean AND
	boolean OR

• ASSUME:

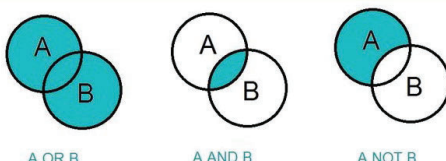
A = 1, B = 2, C = 3, D = 0, F = "Maybe"

LOGICAL TESTS:

Result = A >= B;
= FALSE

Result = ((A >= B) && (C > B)) || (D == 0);
= (FALSE && TRUE) || TRUE;
= TRUE

Result = (A >= B) && ((C > B) || (D == 0));
= FALSE && (TRUE || TRUE);
= FALSE



Use parentheses carefully

Consider holding each logical test in a variable to help debugging

LM@UQ WS7 - Fiji Macros | Aug 2026

Conditional Code (Booleans)

True or False Statements

- A Boolean expression is a 'True' or 'False' statement (may also be referred to as 'on/off' or 'yes/no')

TRUE is assigned a value of 1

FALSE is assigned a value of 0

Using a dialog box with check boxes?

Then, `Checked == 1` is a True statement,
`Unchecked == 1` is a FALSE statement and
`Unchecked == 0` is a True statement

- An unlimited number of expressions can be combined in your macro

Remember operators? Combine Boolean expressions using `&&` for "and" and `||` for "or"

A	B	A&&B	A B	A==B	A!=B
0	0	0	0	1	0
0	1	0	1	0	1
1	0	0	1	0	1
1	1	1	1	1	0

```
x = 5;
y = 10;

Test1 = y>x;           // Is TRUE
Test2 = y == [x*2];    // Is TRUE
Test3 = Test1 && Test2; // Is TRUE
Test4 = y>x && y == [x*3]; // Is FALSE
```

Lack of parentheses makes this *difficult* to read!

Booleans are useful for loops, if variable is less than `nResults` then do XYZ

LM@UQ WS7 - Fiji Macros | Aug 2026

Getting date and time information

Useful Functions

Being able to report when code was run either the time or the date is very useful information you should capture.



Collecting generic date and time information can be achieved with the following code:

```
getDateAndTime(year, month, dayOfWeek, dayOfMonth, hour, minute, second, msec);
```



When wanting to time how long something runs use the following built in command to get the time in milliseconds:

```
getTime();
```



LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 2: Logging General Info

Open a new FIJI script to start this exercise

Things like date and time is useful for output directories, script run date info and confirming script runtime.

5
Minutes

Task:

Sometimes you want to collect 'general' information and report it out to the log window.

- Report the current date (day/month/year) and time (hh:mm:ss) to the log window on separate lines
- Report in seconds how long it took to run

Hint:

Define a variable iStartTime=..... and later report a calculated result...

One of the variables for date & time behaves differently to the others... check your work!

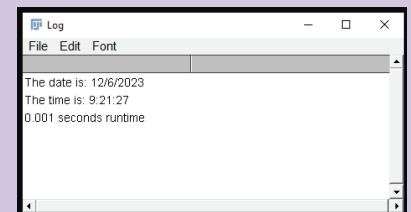


Exercise 2 - Answers

1. Clear the log window
2. Define a variable and collect the time in milliseconds
3. Get the date and time in individual variables
4. Print the formulated date
5. Print the formulated time
6. Calculate run time (current – start)
7. Print the calculate end time as seconds (not milliseconds)

****Note month is base0
(January = 0, February = 1, etc)**

Extended answer contains a function that pads 0's to single character dates e.g. 19/8/2024 becomes 19/08/2024



```
Exercise2_Answers.ijm  New_.ijm
1 //resets log window
2 print("\Clear");
3
4 //get the current time (ms)
5 iStartTime = getTime();
6
7 //get the current date and time
8 getDateAndTime(year, month, dayOfWeek, dayOfMonth, hour, minute, second, msec);
9
10 //print the date and time
11 print("The date is: "+dayOfMonth+"/"+(month+1)+"/"+year);
12 print("The time is: "+hour+": "+minute+": "+second);
13
14 //get the current time (ms)
15 iEndTime = getTime();
16 print((iEndTime-iStartTime)/1000+" seconds runtime");
```

4

Programming Basics – Conditional Code

Adding some logic to your
macros

Operations - Advanced

Increments

Operator	Description
++	pre or post increment
--	pre or post decrement

• ASSUME:

A = 1, B = 2, C = 3, D = 0, F = "Maybe"

INCREMENT:

- Result1 = C * D;
= 0
- Result2 = B + D++;
= 2
- Result3 = C * D;
= 3

- Increment is most frequently encountered in the syntax for loops...*more on this shortly*

IN PLAIN ENGLISH...

AFTER you use the current value of D (i.e. 0) increment it by one e.g. (D now = 1)

Operations - Advanced

Decrement

Operator	Description
++	pre or post increment
--	pre or post decrement

• **ASSUME:**

A = 1, B = 2, C = 3, D = 0, F = "Maybe"

DECREMENT:

Result1 = C*D;
= 0

Result2 = B + --D;
= 1

Result3 = C*D;
= -3

IN PLAIN ENGLISH...

BEFORE you use the current value of D (i.e. 0) decrement it by one (-1)

Conditional code blocks

Also known as loops

IF

- Code block will run once if the condition is met
- (Optional) elseif code block(s) will run once if they are true, in order of appearance
- (Optional) else signals the last 'default' IF code block

n = 1

For

- Code block will run for a number of times
- Number of runs can be user defined (static) or measurement dependant (dynamic)

n = N times

While

- Code block will run for an unlimited amount of times while a condition is true.
- Great for when you don't know how long or many times to run the code block for.

n = X times until condition met

Conditional code blocks

if statements

- **This type of conditional code:**

Begins with the **if** keyword

Uses parentheses () to enclose a logical statement (a.k.a. condition)

Code that will run is contained within braces

- {}, a.k.a. "squiggly brackets"
- **Pro Tip:** Use indenting to visually separate nested levels of code

Only runs when a logical condition evaluates to TRUE

```
if (condition) {
    // DO SOME CODE
}
```

Condition is FALSE, code block will not run!

```
nImages = 0;
if (nImages>0) {
    title = getTitle();
    print("title: " + title);
}
```

Function operates on the currently open window, so it will crash **if** no images have been opened by the user!

LM@UQ WS7 - Fiji Macros | Aug 2026

Conditional code blocks

else / elseif

- **This type of conditional code:**

Uses the **else** keyword to run different code when the condition for **if** evaluates to FALSE

Can chain **elseif** blocks with different condition

- Will **sequentially** check **for the first** TRUE statement
- **Pro Tip:** Avoid ambiguity and races between your conditional statements

```
if (condition) {
    // DO CODE BLOCK
}
else {
    // DO DEFAULT CODE
    // Warn the user?
}
```

```
if (condition1) {
    // DO CODE BLOCK 1
}
elseif (condition2) {
    // DO CODE BLOCK 2
}
else {
    // DO DEFAULT CODE
    // Warn the user?
}
```

```
nImages = 4;

if (nImages==0) {
    print("ERROR: No image windows are open!");
}
elseif (nImages > 0) {
    // THIS BLOCK WILL RUN FIRST
    print("I have at least ONE image!");
}
elseif (nImages == 4) {
    // THIS BLOCK WILL NEVER RUN!
    print("I have exactly FOUR images!");
}
else {
    print("I give up");
}
```

LM@UQ WS7 - Fiji Macros | Aug 2026

Conditional code blocks

for

- **This type of conditional code:**

Begins with the **for** keyword

Uses parentheses () to enclose:

- Initializing a counter
- Checking a continuation condition
- Incrementing the counter

Code within braces gets executed

- Typically, the code uses the loop index to do something different with each iteration

Or instead of nResults

longArraylen = longArray.length

LM@UQ WS7 - Fiji Macros | Aug 2026

```
for (initialization; condition; increment)
{
    // DO CODE BLOCK
}
```

```
for (i=0; i<10; i++) {
    j = 10*i;
    print(j);
}
```

i= 0 j= 00

User-defined (10 iterations)

```
for (idx=0; idx<nResults; idx++) {
    print(getResult("Area", idx));
}
```

Measurement-dependant condition
(i.e. number of results)

Conditional code blocks

while

- **This type of conditional code:**

Begins with the **while** keyword

Uses parentheses () to enclose:

- A continuation condition

Code within braces gets executed

- Typically, the value of the variable for the continuation condition gets checked/modified before next iteration

Best to use a **while** loop when

- You do not know when to stop
- First success/failure of code block is a stopping criterion

```
while (condition)
{
    // DO CODE BLOCK

    // UPDATE CONDITION
}
```



```
finished = FALSE;
while (finished != TRUE) {
    if (can-see-bottom-of-bowl) {
        finished = TRUE;
    }
    else {
        keepating(icecream);
    }
}
```



LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 3: Using Loops

Open a new FIJI script to start this exercise, and the provided example Image file.

5
Minutes

Task:

Using a for Loop, iterate over a Z-stack, print the current slice number for the given Z-stack.

Hint:

```
print("Current Slice ..." _____ );
nSlices      //reports the number of slices
in the current window
```

Bonus: Recreate the example code using for loop

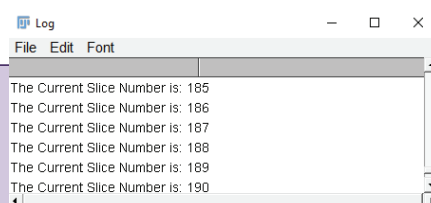
Example Code:

```
while (nImages>0) {close();}
```

LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 3 - Answers

1. Create a loop that will run for each slice (ie for `nSlices()` ;
2. Within that loop, print the current loop number (i+1; remember Base-0) with a text description using the `print()` ; command.
3. Bonus: Work backwards from the total number of images (i--) until there are no more images (0).
4. Set `i=nImages()` ; .and the loop to run each time i is greater than 0, and each time the loop runs, reduce i (i--).
5. For each loop run the `close()` ; command.



```
1 //resets Log window
2 print("\Clear");
3
4 //FOR LOOP, starting at base 0, note less than nSlices
5 for (i = 0; i < nSlices; i++) {
6     //printed defined string + calculated loop integer (for base 0)
7     print("The Current Slice Number is: +(i+1));
8 }
9
10 //FOR LOOP, starting at base 1, note less than equal to nSlices
11 for (i = 1; i <= nSlices; i++) {
12     //printed defined string + loop integer
13     print("The Current Slice Number is: +i);
14 }
15
17 //BONUS ANSWER
18 //reverse Loop starts high, ends at 0; subtracts each iteration
19 for (i=nImages; i>0; i--){
20     close();
21     print("Number of windows left open = " + i);
22 }
23 print("All windows closed");
24
```

LM@UQ WS7 - Fiji Macros | Aug 2026

5 Scripting in Fiji

Introduction & core
concepts

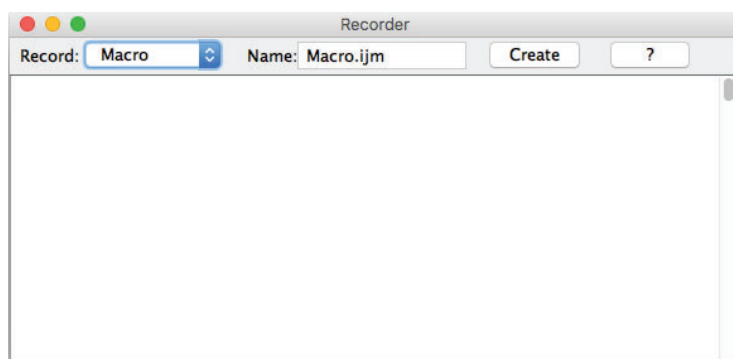
Getting the right commands

Macro Recorder is a really useful tool
[Plugins ► Macro ► Record]

**You probably already know the steps on HOW
to do something in Fiji, but what is the code
needed to do it?**

The macro recorder will display all the steps you
do to your images in code form.

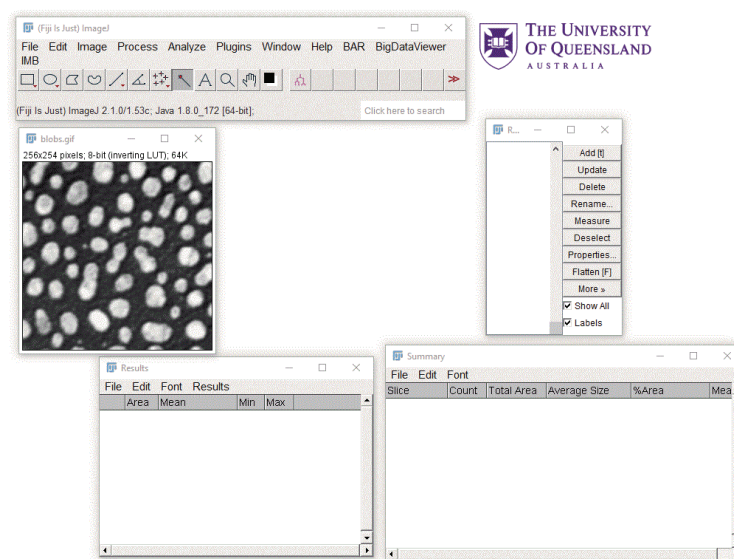
It's good for when you know how to do something
but not what's its called.



Example nuclei quantification workflow

Detect nuclei, measure intensity

1. Duplicate image
2. Filter (blur) image
3. Segment Objects
4. Determine object of interest size
5. Analyze particles
6. Measure intensity in original window
7. Export results to 3rd party program for quantification



LM@UQ WS7 - Fiji Macros | Aug 2026

Commands from the recorder

No everything will be suitable to include in the final code....

```

1 run("Blobs (25K)");
2 run("Invert");
3 run("Duplicate...", " ");
4 run("Median...", "radius=2");
5 setAutoThreshold("Default dark");
6 //run("Threshold...");
7 setAutoThreshold("Moments dark");
8 //setThreshold(0, 127);
9 setOption("BlackBackground", false);
10 run("Convert to Mask");
11 run("Measure");
12 run("Analyze Particles...", "size=50-Infinity show=Masks exclude clear summarize add");
13 selectWindow("Mask of blobs-1.gif");
14 roiManager("Select", 8);
15 run("Select All");
16 roiManager("Select", newArray(0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,:
17 run("Set Measurements...", "area mean min feret's redirect=None decimal=3");
18 roiManager("Measure");
19 saveAs("Results", "C:/Users/uqdbarka/Desktop/test/Akoya/daktest/Results.csv");
20

```

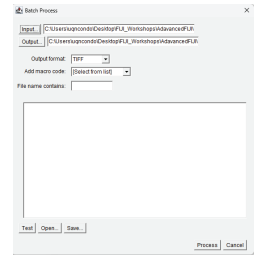
LM@UQ WS7 - Fiji Macros | Aug 2026

Simple Workflows don't need full macros

Batch Processing

This tool is great for linear workflows where you are likely to only need one image window.

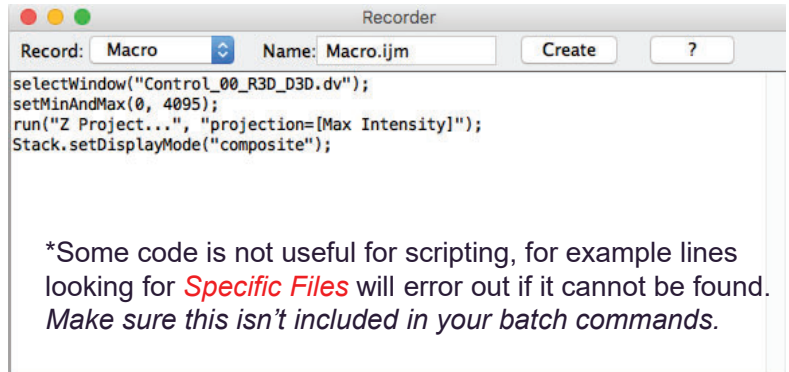
An example of this would be to batch Max Project a number of images.



Batch Processing handles open/save/close separately

Steps

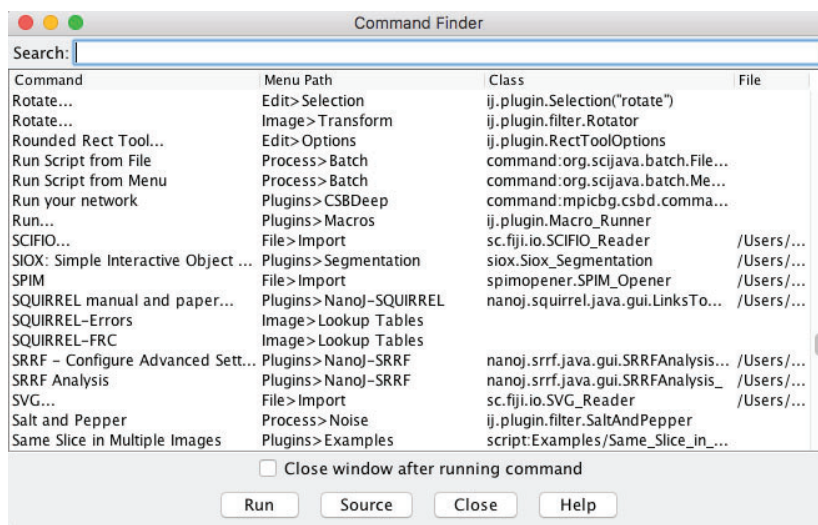
1. Reset Brightness & contrast
2. Maximum Project (Z)
3. Display as a composite



Finding the right commands

Command Finder

[Plugins ► Utilities ► Command Finder]



- Searchable
- Shows menu path
- Executable by clicking
- Links to source WIKI pages online

Finding the right commands

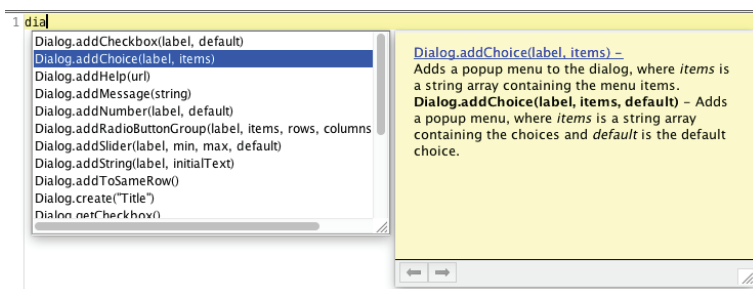
Script Editor V2

[File ► New ► Script]

In the newer versions of Fiji the script editor has autocomplete which makes it easier

The pop-up shows the command, a description and links to the full web description

It also knows variables you have stored so you don't make silly spelling or case mistakes



LM@UQ WS7 - Fiji Macros | Aug 2026

It even helps with loops...

```
for
for (i = 0; i < 10; i++) {
for (i = 0; i < nResults(); i++) { v = getResult('Area', i); set
for (i = 1; i <= nSlices; i++) { setSlice(i); // do something h
```

Common commands

Fiji applies functions to the image itself, so you have to select/handle the image window to apply a function to it.

`getTitle();` - e.g. `windowtitle = getTitle();`

`selectWindow();` - e.g. `selectWindow(windowtitle);`

`close();` //Closes the currently selected window

`run("Measure");` //Runs the measure command on the current selection or window

`run("Clear Results");` //Deletes any data currently stored in the Results Table

LM@UQ WS7 - Fiji Macros | Aug 2026

Scripting: Function outputs

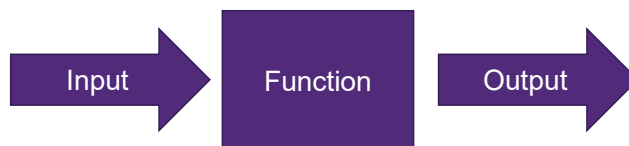
Assigning function outputs to variables that you can use later

No Arguments

```
getTime();  
iTime = getTime();
```

Single Arguments

```
roiManager("count");  
iNuc = roiManager("count");
```



However sometimes in FIJI:

```
getPixelSize(unit, pixelWidth, pixelHeight);
```

Assigns the pixel unit to variable unit, the pixel width to variable pixelWidth, and the pixel height to variable pixelHeight

LM@UQ WS7 - Fiji Macros | Aug 2026

Customising your commands

Arguments can be joined as long as they are the correct data type

```
usrChosenMedian = 3;  
run("Median...", "radius="+usrChosenMedian);
```

What FIJI interprets:

```
run("Median...", "radius=3");
```

```
//Creates Directory for output images/logs/results table  
path=getDirectory("Choose your directory");  
[user chooses Desktop\MyData]  
getDateAndTime(year, month, week, day, hour, min, sec, msec);  
resultsDir = path+"_Results_"+"_"+year+"-"+(month+1)+"-"+day+"\n";  
File.makeDirectory(resultsDir);
```

What FIJI interprets:

```
File.makeDirectory("C:\Desktop\MyData\_Results__2021-11-8\");
```

LM@UQ WS7 - Fiji Macros | Aug 2026

Nesting commands

If the result of a command can be directly parsed into another command it doesn't need its own variable.

For example to print the "Area" from the first row of the results table you could write out:

```
myResult = getResult("Area",0);  
print(myResult);
```

These commands could however be nested to be:

```
print(getResult("Area",0));
```

Note the nested command doesn't have a semi-colon as to not prematurely terminate the command & note the paired brackets to ensure proper syntax.

Getting information from an Image

There are a few different commands to extract different information about the content of image windows.

```
getTitle();  
getDimensions(width, height, channels, slices, frames);  
nSlices;  
getPixelSize(unit, pixelWidth, pixelHeight);  
getVoxelSize(width, height, depth, unit);
```

Note that these commands declare a variable then populate it with result. Some of these repeat themselves, so you can *rename* the variable within the command.

For example

```
getVoxelSize(voxWidth, voxHeight, voxDepth, voxUnit);
```

Exercise 4: Getting info from an image

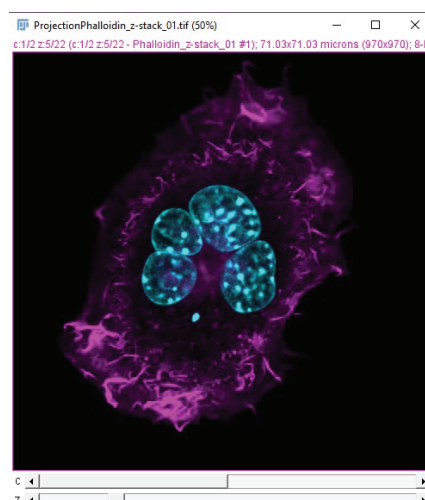
Open a new FIJI script to start this exercise, and the example image file provided in the folder.

10
Minutes

Task:

Use the relevant commands to get & print the following information from the image file:

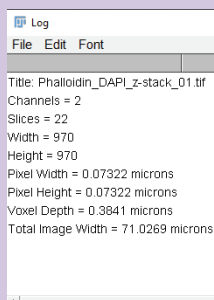
- File name
- Number of channels
- Number of slices
- Image Width (numPix)
- Image Height (numPix)
- Pixel Width + unit
- Pixel Height + unit
- Voxel Depth + unit
- Bonus: Calculate Total Image Width (in microns)



LM@UQ W57 - Fiji Macros | Aug 2026

Exercise 4 - Answers

1. Declare a variable & populate it with the image name using `getTitle()`
2. Use the command `getDimensions(width, height, channels, slices, frames)` print the answers out individually.
3. Use the command `getPixelSize(unit, pixelWidth, pixelHeight)`, print the answers out individually
4. Use the command `getVoxelSize(voxWidth, voxHeight, unit)`, print the answer out individually
5. Calculate `totalImageWidth` by multiplying `pixelWidth` by image width & then print out the answer



```
*Exercise4_Answers.ijm
1 //resets log window
2 print("\nClear");
3
4 //Gets window filename
5 imageTitle = getTitle();
6 print("Title: "+imageTitle);
7
8 //Gets dimensions of the image file
9 getDimensions(width, height, channels, slices, frames);
10 print("Channels = "+channels);
11 print("Slices = "+slices);
12 print("Width = "+width);
13 print("Height = "+height);
14
15 //Gets scale information from the image file (2D)
16 getPixelSize(unit, pixelWidth, pixelHeight);
17 print("Pixel Width = "+pixelWidth+" "+unit);
18 print("Pixel Height = "+pixelHeight+" "+unit);
19
20 //Gets scale information from the image file (3D)
21 getVoxelSize(voxWidth, voxHeight, voxDepth, unit);
22 print("Voxel Depth = "+voxDepth+" "+unit);
23
24 //Calculates the image size (scale not pixel)
25 totalImageWidth = pixelWidth * width;
26 print("Total Image Width = "+totalImageWidth+" "+unit);
27
```

LM@UQ W57 - Fiji Macros | Aug 2026

Getting information about the current state

Sometimes you need information about FIJI and what is running.
Or information about the date and time.

```
getVersion();           //prints ImageJ/FIJI version
nImages                 //Prints/reports the number of images open
getTime();              //Prints/reports the time as integer in ms
getDateAndTime(year, month, dayOfWeek, dayOfMonth, hour, minute, second, msec);
```

Note month is Base-0, rest are Base-1

```
getDateAndTime(year, month, dayOfWeek, dayOfMonth, hour, minute, second, msec);
print(month);
print(month+1);
print("For Context today's date is "+dayOfMonth + "/" + (month+1) + "/" + year );
```

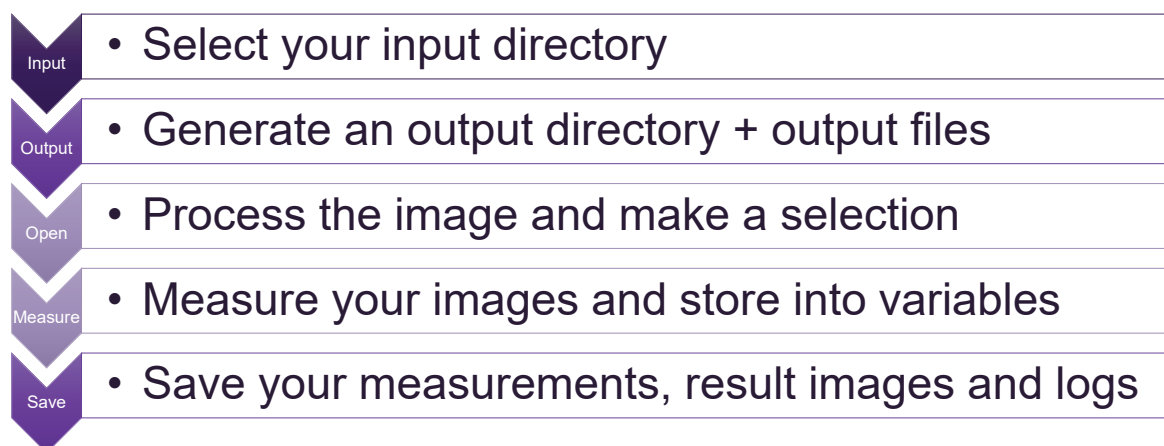
LM@UQ WS7 - Fiji Macros | Aug 2026

6 Structuring your macros

How to develop an image
analysis pipeline

Scripting in Fiji

Generally for scripts you want to follow a defined and well thought out pipeline:



LM@UQ WS7 - Fiji Macros | Aug 2026

Resetting your workspace

Its important to have a *clean* workspace when you run a script/macro. This means no extra windows open, no results left in the results table, no errant ROIs stored in the manager...

```
while(nImages>0){close();} //loop that keeps
                           closing windows until
                           no more are left

run("Clear Results");      //clears the results
                           table

roiManager("reset");       //deletes anything in
                           the ROI manager
```



LM@UQ WS7 - Fiji Macros | Aug 2026

File paths

A **file path** tells the program where abouts to look for the images you want to process.

For example, if the image was in a folder on the desktop called "Images" the path would look like:

C:\Users\<usrName>\Desktop\Images\Image.tif

Importantly the direction of the slash is OS dependant. If defining a specific path use

```
fs = File.separator;
path = "C:"+fs+"usrName"+fs+"Desktop"+fs+"Images"+fs+"Image.tif";
print(path);
```

or

```
getDirectory("Choose Your Directory"); //Displays system dialog to find files.
```

LM@UQ WS7 - Fiji Macros | Aug 2026



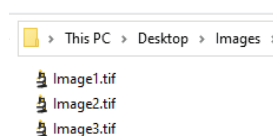
Getting the list of files in your directory

Once you define where your images are, you need to know how many files and their names.

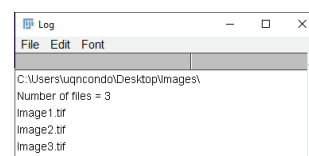
Determine the number of files within directory like this:

```
path = getDirectory("Choose Your Directory");
filelist = getFileList(path);
print(path);
print("Number of files = "+filelist.length);

for (i = 0; i < filelist.length; i++) {
    File.getName(path+filelist[i]);
}
```



```
Path="C:\Users\uqncondon\Desktop\Images\"
filelist=["Image1.tif", "Image2.tif", "Image3.tif"]
Array[ 0 1 2 ]
```



LM@UQ WS7 - Fiji Macros | Aug 2026

Making a new output directory

To keep things tidy, output files should be placed in a different directory (can be within the current directory)

```
path = getDirectory("Choose Your Directory");
resultsDir = path+"_Results";
File.makeDirectory(resultsDir);
```

Note the path will end with a file separator, so the new folder will be called “_Results”

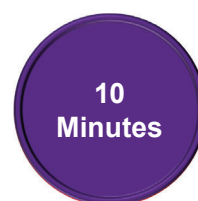
Hint, adding the underscore () will ensure it is at the top of the file list for convenience.

LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 5: File name checking

A short tool to check if files are images

Open a new FIJI script to start this exercise, note you will need a local directory with images in it.



Task:

Define the input directory location

Print out that location

Print out how many files are in it.

Create a results directory inside the input directory.

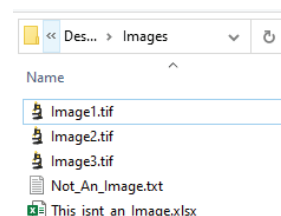
Bonus1: Put the Date & time in the output directory name

Bonus2: Count how many image files are in the input directory

Note: you can match strings with the following commands

```
startsWith(string, prefix);
```

```
endsWith(string, suffix);
```



LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 5: Answers

1. Get the input directory and file list
2. Use .length tool to print the number of files in the file list
3. Create a results directory
4. Define a variable for isImage
5. Run a loop over each file in the file in the list, get its name and query if it ends in .tif, if it does update the isImage variable.
6. Print the number of images found (isImage) to the log window
7. Save the log Window

```

1 path = getDirectory("Choose Your Directory");
2 filelist = getFileList(path);
3 print(path);
4 print("Total Number of files = "+filelist.length);
5
6 getDateAndTime(year, month, dayOfWeek, dayOfMonth, hour, minute, second, msec);
7 resultsDir = path+"_Results_"+year+"-"+(month+1)+"-"+dayOfMonth+"_at_"+hour+"-"+minute+"/";
8 File.makeDirectory(resultsDir);
9
10 isImage = 0;
11
12
13 for (i = 0; i < filelist.length; i++) {
14     currentImage = File.getName(path+filelist[i]);
15     if(endsWith(currentImage, ".tif")) {isImage = isImage+1;}
16 }
17
18 print("There is/are "+isImage+" file(s) ending with tif");
19 selectWindow("Log");
20 saveAs("Text", resultsDir+"Log.txt");
21

```

LM@UQ WS7 - Fiji Macros | Aug 2026

Opening images

To open an image, you first have to tell FIJI where to look for it. If there is more than one image, you need to parse the filename as a variable.

Opening the file could be via inbuilt opener or with a plugin such as Bio-Formats...

```
open(path+filelist[i]);
```

OR

```
run("Bio-Formats Importer", "open="+path+filelist[i]+" autoscale
color_mode=Default rois_import=[ROI manager] view=Hyperstack stack_order=XYZCT");
```

The Bio-Formats opener can be customised with different arguments, see the documentation for full details.

Note, FIJI struggles if your filenames have spaces or *strange* characters such as symbols. Using the [] around a filepath+filename can help with this.

LM@UQ WS7 - Fiji Macros | Aug 2026

Knowing the current image title is important

Many processes and tasks make a new window, ensuring your code runs on the correct window is important for it to run properly.

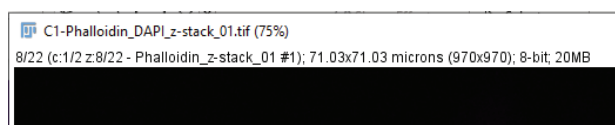
Coupled with the command `selectWindow(windowTitle);` you can confidently know that the tasks you are doing will be completed on the right window in a multistep image analysis process.

For example, see the following code:

```

windowTitle = getTitle();
selectWindow(windowTitle);
run("Split Channels");
selectWindow("C1-"+windowTitle);
rename("Phalloidin"); // a short, more meaningful name for the image

```



LM@UQ WS7 - Fiji Macros | Aug 2026

Saving images

Its important to save not only the final resulting image but the intermediary images to help debug.

To save an image in FIJI, you need to specify the type first (eg .png, .jpg, .tif), followed by the full path.

If you have processed/modified the image, you can append a string to the filename for example:

```

saveAs("Tiff", resultsDir + windowtitlenoext + "_Red-Mask.tif");

```

Note, in order to not end up with a file name called "Filename.tif_Red-Mask.tif"

Use the `replace(string, old, new);` command to string match the file name:

```

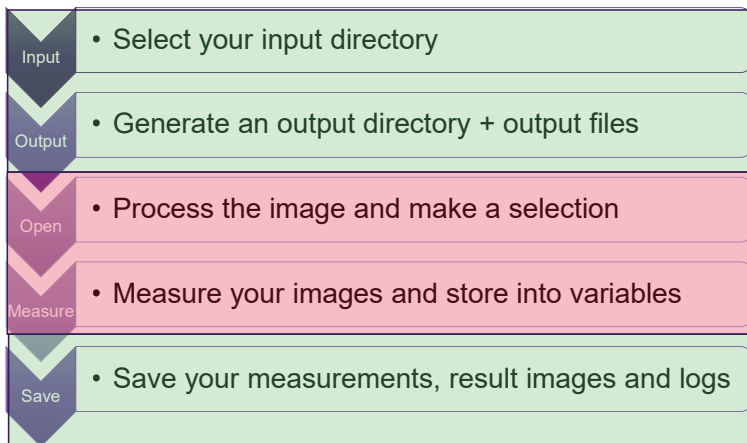
windowtitle = getTitle();
windowtitlenoext = replace(windowtitle, ".tif", "");

```

LM@UQ WS7 - Fiji Macros | Aug 2026

Script shell generator

I have developed a script to generate a script which provides the framework for your image processing



LM@UQ WS7 - Fiji Macros | Aug 2026

```

1 print("\Clear");
2 print("print(\"\\\"\\\"\\\"Clear\\\"");
3 Dialog.create("Details Box");
4 Dialog.addString("Script Auth
5 Dialog.addString("Author Email
6 Dialog.addString("Script Titl
7 Dialog.addString("Version", "
8 Dialog.addString("Description
9 Dialog.addMessage("Note: you
10 Dialog.addString("File Extens
11 Dialog.addCheckbox("Generate
12 Dialog.show();
13 printedAuthor = Dialog.getStr
14 printedAuthorEmail = Dialog.g
15 printedTitle = Dialog.getStri
16 printedVersion = Dialog.getSt
17 printedDescription = Dialog.g
18 ext = Dialog.getString();
19 summary = Dialog.getCheckbox(
20 getDateAndTime(year, month, d
21 if (dayOfMonth<10) {dayOfMont
22 month = month+1;
23 if (month<10) {month = "0" +
24 printedDate = toString(dayOfMonth) + "-" + month + "-" + year;
25 print("");
26 print("// This script has been made with the help of Nicholas Condon's script
27 print("");
28 print("// MIT License");
29 print("// Copyright (c) "+year+" "+printedAuthor+" "+printedAuthorEmail);
30 print("// Permission is hereby granted, free of charge, to any person obtaini
31 print("// of this software and associated documentation files (the \"Software
  
```

Details Box

Script Author: Joe Bloggs

Author Email: j.bloggs@yah

Script Title: Title

Version: 0.0

Description: Description

Note: you can use
 for a line break

File Extension: .tif

☒ Generate Summary File?

OK Cancel

Exercise 6: Using the Script Shell Generator

Create a batch tool to re-colour image LUTs using the Script Shell Generator

10
Minutes

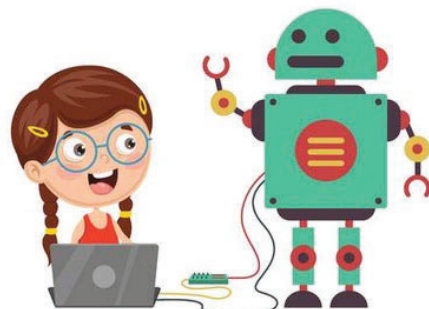
Task:

Run the script shell generator
Copy the text from the log window
Paste it into a new script window

Find the command to change the look up table (LUT) to the FIRE LUT. (**Hint** use the macro recorder on an example image).

Insert the correct command into your new script.

Test and debug

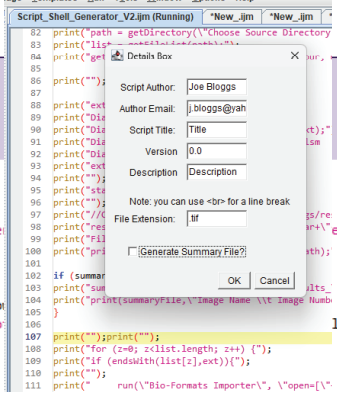


[Image ► LUTS ► Fire]

LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 6: Answers

1. Run the Script Shell Generator and fill in the details as needed
2. Ensure the Generate summary file is **unchecked**
3. Copy the code from the log window into a new script file
4. Open a test image and the macro recorder
5. Change the LUT to Fire and get the command
6. Paste the command into your new script and click run
7. Confirm it runs by checking the output



```

73 for (z=0; z<list.length; z++) {
74   if (endsWith(list[z],ext)){
75
76     run("Bio-Formats Importer", "open");
77     run("Clear Results");
78     roiManager("reset");
79     windowtitle = getTitle();
80     windowtitlenoext = replace(windowtitle, ext, "");
81     print("Opening File: "+(z+1)+" of "+list.length);
82
83     //##### INSERT SCRIPT HERE
84     run("Fire");
85
86
87
88
89
90
91     saveAs("Tiff", resultsDir+ windowtitlenoext+"_output.tif");
92     while(nImages>0){close();}
93   }
94   selectWindow("Log");
95   saveAs("Text", resultsDir+"Log.txt");

```

Printing to a file and not the log window

Sometimes you want to print data not to the log window but to something else such as a spreadsheet.

Here we use `print()` to print to a file (`summaryFile`) rather than the log window.

First declare the new file by creating a variable (eg `summaryFile`);

```
summaryFile = File.open(resultsDir+"Results_Sheet.csv");
```

Next print a title/header for each column:

```
print(summaryFile,"Image Name, Image Number, ROI Number, ROI Area, Ch1 Mean, Ch2 Mean");
```

Then as required, print rows to that file:

```
print(summaryFile,windowtitlenoext+","+i+","+roiNum+","+Area+","+RedMean+","+GreenMean);
```

A note on spreadsheets

There are different possible spreadsheet outputs you can use

- **Comma separated values (CSV) is one of the most common and open formats**

- Values need to be separated by a comma (,)

- In Fiji you would print the comma as a string

- `print(summaryFile,windowtitlenoext+","+i+","+roiNum..`

- **Another common output is a .xls (Microsoft Excel) file**

- Values need to be separated by a tab

- In Fiji you can print a 'tab' by using `\t` as a special reserved term (ie don't use that anywhere else)

- `print(summaryFile,windowtitlenoext+"\t"+i+"\t"+roiNum..`

- You will likely get a warning when opening the excel file output that it wasn't made in excel you can click ok

LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 7: Getting image info

Create a tool which gives you a report on how many channels each image file has.

10
Minutes

Task:

Run the script shell generator

Copy the text from the log window

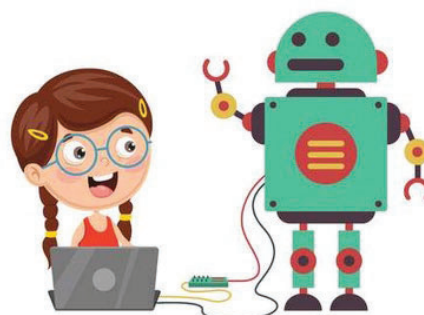
Paste it into a new script window

Add in the commands to get the image file dimensions

Insert the correct commands into your new script.

Update any headers

Test and debug



LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 7: Answers

1. Run the Script Shell Generator and fill in the details as needed
2. Ensure the Generate summary file is **checked**
3. Copy the code from the log window into a new script file
4. Use the get Dimensions command to get how many channels there are in the image window
5. Change the output (header as well) to only one result after image number to match the channels
6. Confirm it runs by checking the output

```
71 summaryFile = File.open(resultsDir+"Results_"+year+"-"+month+"-"+day+"_at_"+hour+"."+min+".xls");
72 print(summaryFile,"Image Name \t Image Number \t Number of Channels");
73
74
75 for (z=0; z<list.length; z++) {
76   if (endsWith(list[z],ext)){
77
78     run("Bio-Formats Importer", "open="+path+list[z]+" autoscale color_mode=Default rois_import=");
79     run("Clear Results");
80     roiManager("reset");
81     windowtitle = getTitle();
82     windowtitlenoext = replace(windowtitle, ext, "");
83     print("Opening File: "+(z+1)+" of "+list.length+" Filename: "+windowtitle);
84
85     //##### INSERT SCRIPT HERE
86     getDimensions(width, height, channels, slices, frames);
87
88     print(summaryFile, windowtitle+"\t"+z+"\t"+channels);
89     saveAs("Tiff", resultsDir+ windowtitlenoext+"_output.tif");
90     while(nImages>0){close();}
91   }
92   selectWindow("Log");
93   saveAs("Text", resultsDir+"Log.txt");
94 }
```

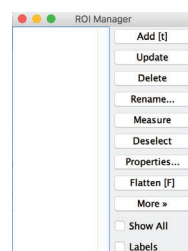
	A	B	C
1	Image Name	Image Number	Number of Channels
2	SingleChannelImage.tif	0	1
3	ThreeChannelImage.tif	1	3
4	TwoChannelImage.tif	2	2
5	UnknownChannelImage.tif	3	10

Region of Interest Manager Commands

The ROI Manager is very useful for storing selections to make measurements, masks, and cropping regions from.

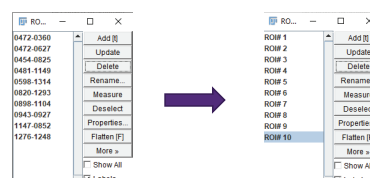
Importantly the ROI manager operates as Base-0

<pre>roiManager("add"); roiManager("delete"); roiManager("select", index); roiManager("measure"); roiManager("rename", name); roiManager("count"); roiManager("reset");</pre>	<pre>//adds selection to roiManager //deletes selection //selects the ROI from the list //measures the ROI selected (or all if none selected) //renames the selected ROI //returns the number of ROIs stored in the manager //deletes all ROIs in the manager</pre>
---	---



For example: Rename each ROI with a number sequence:

```
for (i = 0; i < roiManager("count"); i++) {
  roiManager("select", i);
  roiManager("rename", "ROI# "+(i+1));
}
```



Scaping the results table

Fiji is great at populating the results table with measurements but how do you get that out into the log window or your spreadsheets

Command syntax is as follows: `getResult("column", row);`

The results table is Base-0

```
for (i = 0; i < nResults; i++) {
    print(getResult("Area", i));
}
```

Label	Area	Mean	Min	Max	Feret	Median	FeretX	FeretY	FeretAngle	MinFeret
blobs_gif_copy	281	255	255	255	21.58703	255	195	27	76.60750	18.00000
blobs_gif_copy	75	255	255	255	11.66190	255	134	27	59.03624	9.00000
blobs_gif_copy	271	255	255	255	22.80351	255	217	39	74.74488	16.00000
blobs_gif_copy	230	255	255	255	18.97367	255	43	36	71.56505	16.00000
blobs_gif_copy	496	255	255	255	27.20284	255	170	21	107.10273	24.14953

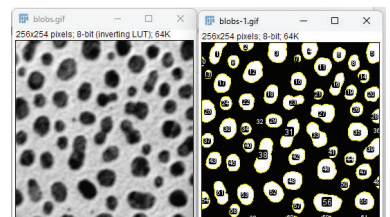
Using arrays to store multiple results

Sometimes due to the nature of the image pipeline you may want to re-calculate the results table but need to report out information in an easy to use way.

Arrays are one dimensional tables you can create and fill with data easily

Imagine the following blobs example...

1. Get the number of results as a variable
2. Create arrays as big as the results table
3. Loop over how many results you have populating your tables
4. Print out to your summary file line by line in a separate loop



```
3 numberBlobs = nResults;
4
5 MeanArray = newArray(numberBlobs);
6 AreaArray = newArray(numberBlobs);
7
8
9 for (i = 0; i < numberBlobs; i++) {
10     MeanArray[i] = getResult("Mean", i);
11     AreaArray[i] = getResult("Area", i);
12 }
13
14 for (i=0; i < numberBlobs; i++) {
15     print(summaryFile, windowtitle+" "+(i+1)+" "+MeanArray[i]+" "+AreaArray[i])
16 }
17
```

Label	Area	Mean	StdDev	Min	Max
blobs.gif.0001-0015	428	191.589	29.704	128	232
blobs.gif.0002-0005	183	179.847	21.329	128	224
blobs.gif.0003-0014	656	205.854	29.110	128	248
blobs.gif.0004-0011	431	217.949	35.404	128	248
blobs.gif.0005-0014	471	213.214	28.533	128	248
blobs.gif.0006-0016	279	205.935	35.178	128	248
blobs.gif.0007-0022	75	164.160	17.663	128	200
blobs.gif.0008-0028	271	176.059	24.643	128	224

Running in batch mode

Sometimes you don't want image windows flashing up and opening when running your scripts

FIJI uses `selectWindow` to select data but presenting data to the computer screen is very slow.

Batch mode allows FIJI to access the image data without presenting the image on the screen

```
setBatchMode(true);
```

Some plugins and tools however won't work in batch mode so if you have issues or errors try disabling this feature.

LM@UQ WS7 - Fiji Macros | Aug 2026

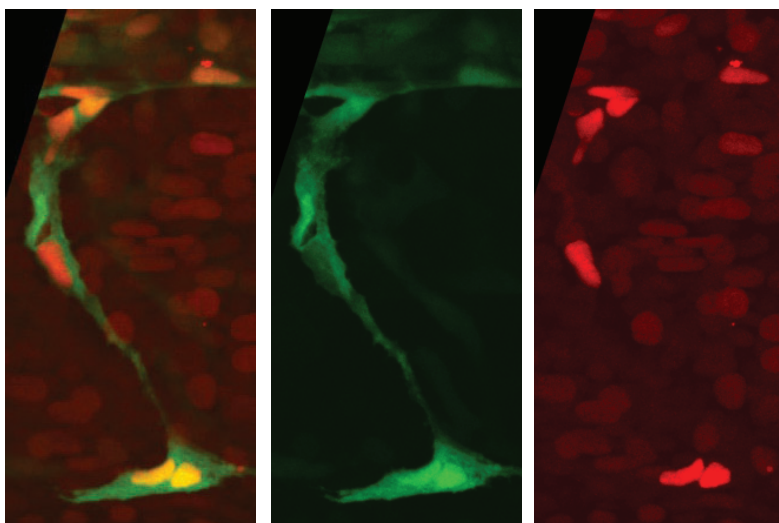
Exercise 8: Multi-dimensional analysis - ROIs

Scenario:

Zebrafish are an important developmental model organism. This fish line expresses a nuclei marker (always on) and a GFP-BioSensor that is sometimes in the cytoplasm, and sometimes in the nuclei.

Create a script to run on a folder of images, to find the nuclei, measure them, measure the nuclei marker intensity level, and the corresponding mean and max intensity of the BioSensor channel.

Save the results to a spreadsheet, along with the mask and nuclei ROIs.



LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 8: Multi-dimensional analysis - ROIs

Open the provided script file for this exercise.

10
Minutes

Task:

This script file has been corrupted!

Use your scripting knowledge to fix the script so it will work once more!

Parts of the script that are corrupted will appear as `#####`

```
// * * * * * Section 05 - Getting measurements
for (i = 0; i < roiManager("count"); i++) {
    selectWindow("nuc");
    run("Clear Results");
    roiManager("select", i);
    roiManager("measure");
    nucArea = getResult("Area",0);
    nucMean = getResult("Mean", 0);

// * * * * * Section 06 - Getting measurements
selectWindow("green");
#####
#####
#####
#####
imageNum = (z+1);
#####
}

// * * * * * Section 07 - Saving & Closing Outp
#####
saveAs("tiff", resultsDir+windowtitlenc
print("Saving nuclei mask");
close();
roiManager("Save", resultsDir+ windowtitlen
print("Saving ROI list");
while(nImages>0){close();}
```

LM@UQ WS7 - Fiji Macros | Aug 2026

Exercise 8: Answers

Using the comments on the right to guide you fill in missing lines of code as follows:

Ln7: `path = getDirectory("Choose Source Directory ");`

Ln28: `open(path+list[z]);`

Ln40: `run("Median...", "radius=4");`

Ln46: `print("Number of Nuclei Found = "+roiManager("count"));`

Ln59: `run("Clear Results");`

Ln60: `roiManager("select", i);`

Ln61: `roiManager("measure");`

Ln62: `ch1Mean = getResult("Mean", 0);`

Ln63: `ch1Max = getResult("Max",0);`

Ln65: `nucNum = (i+1);`

Ln66: `print(summaryFile, windowtitle+" "+imageNum+" "+roiManager("count")+" "+nucNum+" "+nucArea+" "+nucMean+" "+ch1Mean+" "+ch1Max);`

```
48 // * * * * * Section 05 - Getting measurements from
49 for (i = 0; i < roiManager("count"); i++) {
50     selectWindow("nuc");
51     run("Clear Results");
52     roiManager("select", i);
53     roiManager("measure");
54     nucArea = getResult("Area",0);
55     nucMean = getResult("Mean", 0);
56
57 // * * * * * Section 06 - Getting measurements from
58 selectWindow("green");
59 run("Clear Results");
60 roiManager("select", i);
61 roiManager("measure");
62 ch1Mean = getResult("Mean", 0);
63 ch1Max = getResult("Max",0);
64 imageNum = (z+1);
65 nucNum = (i+1);
66 print(summaryFile,windowtitle+" "+imageNum
67 }
68
```

Getting information from the user

Sometimes you need a way to input information into the script for it to run.

The dialog window tool lets users input, strings, numbers, arrays, make single or multiple choices all to help guide the script to run correctly.

First the dialog is created, populated and then shown. See the example below:

```
Dialog.create("Dialog window title");  
    Dialog.addString("Title:", title);  
    Dialog.addChoice("Type:", newArray("8-bit", "16-bit", "32-bit", "RGB" ));  
    Dialog.addNumber("Width:", 512);  
    Dialog.addNumber("Height:", 512);  
    Dialog.addCheckbox("Checked?", true);  
Dialog.show();
```

LM@UQ WS7 - Fiji Macros | Aug 2026

Reading Dialog information into variables

Once some information has been input into the dialog window, the next step is to collect the inputs into variables.

The commands are similar and work on the order of appearance (eg two numbers will scrape in order Width/Height)

After showing the dialog, the user will click ok, and the variables will be populated as per below:

```
Dialog.show();  
Title = Dialog.getString();  
bitDepth = Dialog.getChoice();  
Width = Dialog.getNumber();  
Height = Dialog.getNumber();  
Checked = Dialog.getCheckbox();
```

LM@UQ WS7 - Fiji Macros | Aug 2026

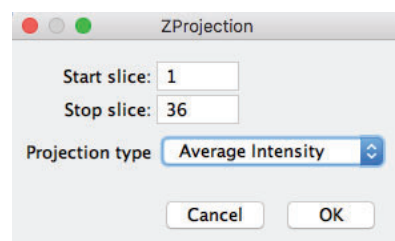
Exercise 9: Re-creating the Z-projection Dialog

In order to test your dialog tools coding knowledge generate the code for the pop-up window for doing the Z-projection function.

10
Minutes

Requirements:

- Default Stop slice = total number of slices
- Dialog Title says Zprojection
- Script exits if no image is open.
- All of the projection types are available
- The option for running time series data or single frame when multiple frames are detected.



Note: You will see an error message when it successfully runs, this is just saying a macro is launching a macro command. Click ok to continue.

LM@UQ WST - Fiji Macros | Aug 2026

Exercise 9: Answers

```
if (nImages()==0) //If no windows are open the script exits
    exit("There is no image open");
else{ //If the active window has been chosen
    getDimensions(width, height, channels, slices, frames); //Gets image dimensions
    Options= newArray("Max Intensity", "Average Intensity", "Sum Slices", "Min Intensity", "Standard Deviation", "Median"); //Creates array

    Dialog.create("Z-Projection"); //Creates a pop-up window
    Dialog.addNumber("Start Slice", 1); //Creates the box for the start slice (Defaults to 1)
    Dialog.addNumber("End Slice", slices); //Creates the box for the end slice (Defaults to the total slice number)
    Dialog.addChoice("Projection Type:", Options); //Creates the drop down list of projection options
    if (frames>1){Dialog.addCheckbox("Project all frames", true);} //Checks if there is more than 1 timepoint and displays a checkbox if there is.
    Dialog.show(); //Shows the dialog box
    startslice = Dialog.getNumber(); //Assigns the user input start slice value to the variable startslice
    endslice = Dialog.getNumber(); //Assigns the user input end slice value to the variable endslice
    projectiontype=Dialog.getChoice(); //Assigns the user input projection type to the variable projectiontype
    Projectallframes = Dialog.getCheckbox(); //Assigns the checkbox option to the variable Projectallframes

    if (Projectallframes == 1){run("%Z Project...", "projection=["+projectiontype+" start="+startslice+" stop="+endslice+" all");} //runs projection.
    else run("%Z Project...", "projection=["+projectiontype+" start="+startslice+" stop="+endslice);} //projects 1 frame.
```

LM@UQ WST - Fiji Macros | Aug 2026

Fiji Coding Quick Guide

Arrays

```
myArray = newArray(0);  
longArray = Array.concat(arr1, arr2)  
Array.print(arr1)
```

Functions

```
path = getDirectory("Choose Your  
Directory");  
filelist = getFileList(path);  
resultsDir = path+"_Results";  
File.makeDirectory(resultsDir);
```

Resetting

```
close();  
run("Clear Results");  
roiManager("reset");  
while(nImages>0){close();}
```

Print statements

```
print();  
print(myResult);  
print(nSlices);  
print("\Clear");
```

LM@UQ WS7 - Fiji Macros | Aug 2026

Conditional statements

```
if (condition = true) {// Do code}  
for (i=0; i<10; i++) {// Do code}  
while (condition) {// Do code}
```

Image information

```
windowTitle = getTitle();  
selectWindow(windowTitle):  
getDimensions(width, height, channels, slices,  
frames);  
nSlices;  
getPixelSize(unit, pixelWidth, pixelHeight);  
getVoxelSize(width, height, depth, unit);  
getResult("column", row);
```

Saving & Reporting

```
saveAs("Tiff", resultsDir+"myfilename");  
summaryFile = File.open(resultsDir+"Results.csv");  
print(summaryFile, "Image Name, ... Results");
```

ROI Manager

```
roiManager("add");  
roiManager("delete");  
roiManager("select", index);  
roiManager("measure");  
roiManager("rename", name);  
roiManager("count");  
roiManager("reset");
```

Common Functions

```
run("Median...", "radius=2");  
run("Z Project...", "projection=[Max  
Intensity]");  
run("Duplicate, ...");  
run("Duplicate...",  
"title="+windowtitle+"_copy");  
run("Split Channels");  
run("Measure");  
myAreaResult = getResult("Area", 0);  
setAutoThreshold("Huang no-reset");  
run("Convert to Mask");
```

7 Coding Tips & Tricks

No one ever sits down and
writes a script that runs
perfectly the first time

Tips & Tricks

Following these will certainly help your future self

- **Always save your intermediate steps**

If you need to prove your work to your boss (or Reviewer #3), generate your thresholds, masked images, regions of interest, and anything else you need **as you go along!**

- **Use well-defined variables**

Var1 is non-descript, consider NucCount1, for example, for Nuclei Count in region 1

- **Liberal comment your scripts**

Describe your variable naming conventions and blocks of code in layman's terms, because other people need to understand them, **including your future self!**

LM@UQ WS7 - Fiji Macros | Aug 2026

Commenting your code

Comments help describe what you are trying to do, and help make the reader understand your approach

- Your scripts should be well documented with comments explaining each section.
- Individual lines of code can be made into a comment (ie ignored by the script) by adding two // at the beginning of the line of code.
- Alternatively larger blocks of code can be ignored by placing /* and */ in lines above and below the code block.

```
//This line of code duplicates  
// the image currently selected  
Run("Duplicate, ...");
```

(Single line Comments)

```
/*  
All of this is ignored  
* And this  
*/  
Run("Duplicate, ...");
```

(Multi-line Comments)

Pseudo coding

Plan out the steps you need to take

- **Writing in words what you want your code to do step by step**
- **Start with the aim of the program**
- **Write out what image processing steps you will need**
- **Then work out the code that achieves your imaging goals**

Pro Tip: Use Macro recorder and Google for code snippets

```
//comment
// aim of this macro is to count the number of cells
// using DAPI channel

User navigates to folder
Read all czi image files in folder
For all files in the folder
    Read in file
    split channels
    select blue DAPI channel
    background subtract
    median filter
    threshold
    analyse particles and add to ROI manager
    count how many are in the ROI manager
    save count to array that is the same length as the
        number of files
Write out file names and array of nuclei counts to excel file
```

Debugging: Knowing your program is working

1. Make it work on one image first, where you have counted the expected result. **Make sure your program can produce the correct result.** Don't try to run your script on the whole directory of images until it works
2. Print statements tell you where your program fails and what your variables are holding
 - Example: say you count 17 nuclei in your image and you save that number to a variable called nucvariable
 - Nucvariable = 17
 - print("I expect to see " + nucvariable + "nuclei")
 - Will show in the log window
 - "I expect to see 17 nuclei"
3. Commenting out lines of code can help you figure out whether it is that command that is breaking your program
 - It allows you to get past a breaking point to see if other parts run.
 - Example:
 - In you code you have **r= 100, g=100, b=400**
 - **setBackgroundColor(r, g, b)**
 - **You get error**
 - **// setBackgroundColor(r, g, b)**
 - Allows the rest of the code to run, even though background colour is not set properly.
 - Solution: b has to be less than 255

Debugging tools

Fiji has a built in debugging tool

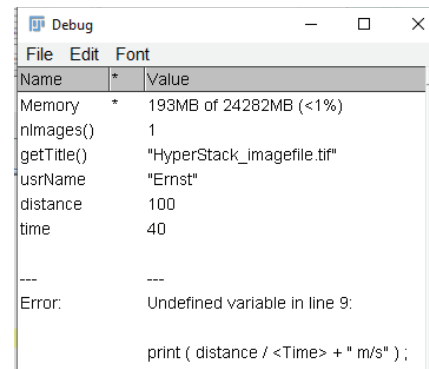
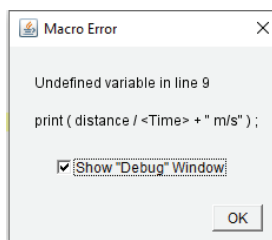
- If you have a mistake in your code FIJI will hit a Macro Error
- for example here a variable is defined as time but called as Time
- The error flags “Undefined variable in line 9”

`print(distance/<Time>+"m/s");`

Note the use of < > around the mistake

- The debug window will show you all the variables stored in memory and the error that was flagged.
- It does sometimes not print properly hiding the error ☹

```
1 print("\Clear");
2 print("Hello World");
3 usrName = "Ernst";
4 print("Hi, my name is "+usrName);
5 print(1990);
6 print(31*12+ " months");
7 distance=100; //in metres
8 time=40; //in seconds
9 print(distance/Time + " m/s");
10 print("");
11 print("My top 3 Austrlian cities are:");
12 cities = newArray("Brisbane", "Sydney", "Melbourne");
13 Array.print(cities);
14 print("My absolute favourite is: "+cities[0]);
```



LM@UQ WS7 - Fiji Macros | Aug 2026

Where to get help

- **Image.sc** is the main website for all things image analysis
 - There are a number of forum categories that should be relevant to your issues
- **GitHub** is a place people store their code which may give you ideas how others approach problems
 - My personal GitHub is <https://github.com/nickcondon>
- A general google search for your problem with terms such as “.ijm” “Fiji macro” “ImageJ Macro” can also be useful
- ChatGPT is also a useful tool for Fiji Macros
- UQ Image Clinic runs Mondays at 11am on Zoom (bit.ly/UQImageClinic)

LM@UQ WS7 - Fiji Macros | Aug 2026



ChatGPT

can you help me make a fiji macro ".ijm" that asks the user where a directory of images is, opens them sequentially and maximum projects each one before saving it in a new results directory

Instructions:

1. Copy the macro code into a text file and save it with the ".ijm" extension.
2. Open Fiji and run the macro.

How it Works:

- `getDirectory("Choose a directory with images to process");`: Prompts the user to select the directory containing images.
- `File.makeDirectory(outputDir);`: Creates a new "Results" directory within the selected folder.
- `getFileList(inputDir);`: Retrieves all files in the selected directory.
- `open(inputDir + list[i]);`: Opens each image file one by one.
- `run("Z Project...", "projection=[Max Intensity]");`: Applies a maximum intensity projection to the image.
- `saveAs("Tiff", outputDir + "MaxProj_" + list[i]);`: Saves the resulting image in the "Results" directory.
- `close("");`: Closes all images to free up memory.

This macro will take each image in the directory, apply a maximum intensity projection, and save the processed image in a subdirectory named "Results".

LM@UQ WS7 - Fiji Macros | Aug 2026



Sure! Below is a Fiji macro (.ijm file) that will ask the user to select a directory containing images, open each image sequentially, perform a maximum intensity projection on each one, and then save the results in a new "Results" directory within the original folder.

```

// Fiji Macro: Maximum Projection of Images in a Directory

// Ask the user to select a directory containing the images
inputDir = getDirectory("Choose a directory with images to process");

// Create a results directory inside the selected directory
outputDir = inputDir + "Results/";
File.makeDirectory(outputDir);

// Get a list of all files in the directory
list = getFileList(inputDir);

// Process each image in the directory
for (i = 0; i < list.length; i++) {
    // Open the image
    open(inputDir + list[i]);

    // Perform maximum intensity projection
    run("Z Project...", "projection=[Max Intensity]");

    // Save the result
    saveAs("Tiff", outputDir + "MaxProj_" + list[i]);

    // Close all images
    close("");
}

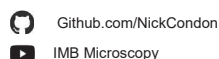
```



Thanks

Dr Nicholas Condon
Senior Principal Imaging Scientist
Institute for Molecular Biosciences
n.condon@uq.edu.au

Special Thanks to
Deb & Kestutis Barkauskas



Feedback

Chan
Zuckerberg
Initiative



Australian Government
Australian Research Council

Australian Government
National Health and Medical Research Council



LM@UQ WS7 - Fiji Macros | Aug 2026